

Django

עבודה על פרוייקט מסכם

TomerBu

זיהוי משתמש אוטומטי בתגובות

לא צריך לשאול משתמש מחובר - איך קוראים לו...

CurrentUserDefault

A default class that can be used to represent the current user. In order to use this, the 'request' must have been provided as of the context dictionary when instantiating the serializer.

```
owner = serializers.HiddenField(  
    default=serializers.CurrentUserDefault()  
)
```

CreateOnlyDefault

A default class that can be used to *only set a default argument during create operations*. During updates the field is omitted.

It takes a single argument, which is the default value or callable that should be used during create operations.

```
created_at = serializers.DateTimeField(  
    default=serializers.CreateOnlyDefault(timezone.now)  
)
```

זיהוי משתמש אוטומטי בתגובות

בקובץ serializers נערוך את הסריילייזר של Comment

בדיוק לפי הדוקומנטציה

```
class CommentSerializer(ModelSerializer):  
    author = serializers.HiddenField(  
        default=serializers.CurrentUserDefault()  
    )  
  
    class Meta:  
        model = Comment  
        fields = '__all__'
```

אצלנו יש UserProfile=>User

זיהוי משתמש אוטומטי בתגובות

בקובץ serializers נערוך את הסרילייזר של Comment

```
class CurrentProfileDefault:
    requires_context = True

    def __call__(self, serializer_field):
        return serializer_field.context['request'].user.userprofile

class CommentSerializer(ModelSerializer):
    author = serializers.HiddenField(
        default=CurrentProfileDefault()
    )

    class Meta:
        model = Comment
        fields = '__all__'
```

קלאס כמו של הספרייה
עם שינוי אחד

אצלנו יש **User** => **UserProfile**

אפשר לעשות ירושה

זיהוי משתמש אוטומטי בתגובות

נוסיף לJSON את הauthor_id

```
class CommentSerializer(ModelSerializer):  
    author = serializers.HiddenField(  
        default=CurrentProfileDefault()  
    )
```

```
author_id = serializers.SerializerMethodField()
```

```
class Meta:  
    model = Comment  
    fields = '__all__'
```

```
def get_author_id(self, obj):  
    return obj.author.id
```

המערכת מאפשרת לנו להוסיף שדות לJSON

זיהוי משתמש אוטומטי בתגובות

נוסיף לJSON את הauthor_id

```
class PostSerializer(ModelSerializer):  
    author = serializers.HiddenField(  
        default=CurrentProfileDefault()  
    )  
    author_id = serializers.SerializerMethodField()  
  
    class Meta:  
        model = Post  
        fields = '__all__'  
  
    def get_author_id(self, obj):  
        return obj.author.id
```

אותו דבר כמעט בPostUserLikes:

הפעם במחלקה קוראים לזה user ולא author

```
class PostUserLikesSerializer(ModelSerializer):
    user = serializers.HiddenField(
        default=CurrentProfileDefault()
    )
    user_id = serializers.SerializerMethodField()

    class Meta:
        model = PostUserLikes
        fields = '__all__'

    def get_user_id(self, obj):
        return obj.user.id
```


אותו דבר כמעט ב:UserProfileSerializer

הפעם במחלקה קוראים לזה user ולא author

```
class UserSerializer(ModelSerializer):  
  
    user = serializers.HiddenField(  
        default=serializers.CurrentUserDefault()  
    )  
  
    user_id = serializers.SerializerMethodField('get_user_id')  
  
    def get_user_id(self, obj):  
        return obj.user.id  
  
    # ....
```


הגדרות של כתובות צד לקוח גם ל-CSRF:

settings.py

```
CSRF_TRUSTED_ORIGINS = [  
    "https://example.com",  
    "https://sub.example.com",  
    "http://localhost:5173",  
    "http://127.0.0.1:5173",  
]
```

ספריה לעבודה עם .env

<https://pypi.org/project/python-dotenv/>

<https://pypi.org/project/python-decouple/>

Usage

Install:

```
pip install python-decouple
```

Then use it on your `settings.py`.

1. Import the `config` object:

```
from decouple import config
```

2. Retrieve the configuration parameters:

```
SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
EMAIL_HOST = config('EMAIL_HOST', default='localhost')
EMAIL_PORT = config('EMAIL_PORT', default=25, cast=int)
```

ספריה לעבודה עם .env

ניצור קובץ .env בתיקיה הראשית

```
.env
1 DB_NAME=drf_blog
2 DB_USER=postgres
3 DB_PASSWORD=o2iefjho3sfhoi32hfoi2fhioh
4 DB_HOST=localhost
5 DB_PORT=5432
6 DEBUG=True
7 SECRET_KEY=507*n29c3+rjya*9-+1=n=6*#p!mc%zfog1@sv7d0yp^g=8q8c
```

נריץ פקודה בטרמינל כדי לקבל רנדום - או נבקש מ LLM

```
python -c 'from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())'
```

ספרייה לעבודה עם .env

```
from decouple import config

DB_NAME = config('DB_NAME')
DB_USER = config('DB_USER')
DB_PASSWORD = config('DB_PASSWORD')
DB_HOST = config('DB_HOST')
DB_PORT = config('DB_PORT')

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': DB_NAME,
        'USER': DB_USER,
        'PASSWORD': DB_PASSWORD,
        'HOST': DB_HOST,
        'PORT': DB_PORT,
    }
}
```

ספריה לעבודה עם .env

```
from decouple import config
```

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': config('DB_NAME'),  
        'USER': config('DB_USER'),  
        'PASSWORD': config('DB_PASSWORD'),  
        'HOST': config('DB_HOST'),  
        'PORT': config('DB_PORT'),  
    }  
}
```

ספריה לעבודה עם .env

```
# SECURITY WARNING: keep the secret key used in product
SECRET_KEY = config('SECRET_KEY')
# python -c 'from django.core.management.utils import g
# SECURITY WARNING: don't run with debug turned on in p
DEBUG = config('DEBUG', default=True, cast=bool)
```

בהמשך נכניס גם את הערכים הללו לenv.

```
CORS_ALLOWED_ORIGINS = [  
    "https://example.com",  
    "https://sub.example.com",  
    "http://localhost:5173",  
    "http://127.0.0.1:5173",  
]  
  
CSRF_TRUSTED_ORIGINS = [  
    "https://example.com",  
    "https://sub.example.com",  
    "http://localhost:5173",  
    "http://127.0.0.1:5173",  
]
```

**ממילא נצטרך לחזור לזה כי אין לנו כרגע כתובות של השרת
האמיתי...**

תיקון לSerializers:

```
class UserSerializer(ModelSerializer):
    user = serializers.HiddenField(
        default=serializers.CurrentUserDefault()
    )
    user_id = serializers.SerializerMethodField('get_user_id')

    password = serializers.CharField(
        validators=[
            RegexValidator(
                regex=r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*]).{8,}$',
                message="Password must be at least 8 characters long and contain at least one letter and one number."
            )
        ],
        write_only=True
    )

    def get_user_id(self, obj):
        return obj.id

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user

    def update(self, instance, validated_data):
        # remove the password from the dictionary
        password = validated_data.pop('password', None)

        # iterate over the dict and set the attributes of the user instance
        for key, value in validated_data.items():
            setattr(instance, key, value)

        instance.set_password(password)
        instance.save()
        return instance

class Meta:
    model = User
    fields = ['id', 'username', 'email', 'password', 'user_id', 'user']
    extra_kwargs = {'password': {'write_only': True}}
```


DRF Throttling

הגבלת כמות הכניסות למשתמש לא רשום

הגבלת כמות הכניסות למשתמש רשום

אפשר ליישם:

ברמת הView

וגם ברמת ברירת מחדל

משתמש בכתובות IP

לקוח שקיבל הגדרה 6 בקשות ביום - יכול לקבל 6 בקשות באתר.

DRF Throttling

הגדרה גלובלית

```
REST_FRAMEWORK = {  
    'DEFAULT_THROTTLE_CLASSES': [  
        'rest_framework.throttling.AnonRateThrottle',  
        'rest_framework.throttling.UserRateThrottle'  
    ],  
    'DEFAULT_THROTTLE_RATES': {  
        'anon': '100/day',  
        'user': '1000/day'  
    }  
}
```

הגדרה ברמת הView

```
from rest_framework.response import Response  
from rest_framework.throttling import UserRateThrottle  
from rest_framework.views import APIView
```

```
class ExampleView(APIView):  
    throttle_classes = [UserRateThrottle]  
  
    def get(self, request, format=None):  
        content = {  
            'status': 'request was permitted'  
        }  
        return Response(content)
```

הגדרה ל
function based view

```
@api_view(['GET'])  
@throttle_classes([UserRateThrottle])  
def example_view(request, format=None):  
    content = {  
        'status': 'request was permitted'  
    }  
    return Response(content)
```

קיימת גם הגדרה במיוחד ל
custom actions

DRF Throttling

```
'DEFAULT_THROTTLE_CLASSES': [  
    'rest_framework.throttling.AnonRateThrottle',  
    'rest_framework.throttling.UserRateThrottle'  
],  
'DEFAULT_THROTTLE_RATES': {  
    'anon': '1/min',  
    'user': '5/minute'  
}
```

DRF Throttling

Django REST framework

Api Root

Api Root

The default basic root view for DefaultRouter

GET /api/

HTTP 429 Too Many Requests

Allow: GET, HEAD, OPTIONS

Content-Type: application/json

Retry-After: 52

Vary: Accept

{

"detail": "Request was throttled. Expected available in 52 seconds."

}

DRF Throttling

```
'DEFAULT_THROTTLE_CLASSES': [  
    'rest_framework.throttling.AnonRateThrottle',  
    'rest_framework.throttling.UserRateThrottle'  
],  
'DEFAULT_THROTTLE_RATES': {  
    'anon': '1/min',  
    'user': '5/minute'  
}
```

DRF Throttling

```
from api.throttles import MyRateThrottle

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
    permission_classes = [TagsPermission]
    throttle_classes = [MyRateThrottle]
```

DRF Throttling

```
from rest_framework import throttling
from rest_framework.throttling import SimpleRateThrottle
from django.core.cache import cache as django_cache

# cache is ~ dictionary with a time-to-live (TTL) for each key

class MyRateThrottle(throttling.BaseThrottle):
    def allow_request(self, request, view):
        user = request.user
        username = user.username

        if not django_cache.get(username):
            django_cache.set(username, 0, timeout=60)

        django_cache.incr(username)

        return django_cache.get(username) <= 10
```


הוספת יכולות סינון

DjangoFilterBackend

The `django-filter` library includes a `DjangoFilterBackend` class which supports REST framework.

To use `DjangoFilterBackend`, first install `django-filter`.

```
pip install django-filter
```

Then add `'django_filters'` to Django's `INSTALLED_APPS`:

```
INSTALLED_APPS = [  
    ...  
    'django_filters',  
    ...  
]
```

You should now either add the filter backend to your settings:

```
REST_FRAMEWORK = {  
    'DEFAULT_FILTER_BACKENDS': ['django_filters.rest_framework.DjangoFilterBackend'],  
}
```


הוספת יכולות סינון

```
pip install django-filter
```

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'api',  
    'drf_spectacular',  
    "corsheaders",  
    'django_filters'  
]
```

הוספת יכולות סינון

```
REST_FRAMEWORK = {  
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',  
  
    'DEFAULT_PERMISSION_CLASSES': [  
        'rest_framework.permissions.IsAuthenticated',  
    ],  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework.authentication.SessionAuthentication',  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
  
    'DEFAULT_THROTTLE_CLASSES': [  
        'rest_framework.throttling.AnonRateThrottle',  
        'rest_framework.throttling.UserRateThrottle'  
    ],  
    'DEFAULT_THROTTLE_RATES': {  
        'anon': '1/min',  
        'user': '5/minute'  
    },  
    'DEFAULT_FILTER_BACKENDS': [  
        'django_filters.rest_framework.DjangoFilterBackend'  
    ],  
}
```

הוספת יכולות סינון

```
class PostViewSet(ModelViewSet):  
    queryset = Post.objects.all()  
    serializer_class = PostSerializer  
    permission_classes = [PostsPermission]  
    filterset_fields = ['title', 'text', 'author_id']
```

localhost:8000/api/posts/?title=ViewSets&text=&author_id=

kam... SF | Priza-Net (v. 4.5) - Designing INSTEA... SQL SERVER - @... COLUMNS_UPDA... Decode COLUMN... Using Columns_U... Introduction to SQ... stanf S Toolbar Animation... Handling Scrolls w... fitsSystemWindow...

Django REST framework

Api Root / Post List

Post List

GET /api/posts/?title=ViewSets&text=

HTTP 200 OK

Allow: GET, POST, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

Filters

×

Field filters

Title: ViewSets

Text:

Author: -----

▼

Submit

Filters

הוספת יכולות סינון

http://localhost:8000/api/posts/?title=ViewSets&text=&author_id=



פרוטוקול

http://localhost:8000/api/posts/?title=ViewSets&text=&author_id=

host

דומין או ip

http://localhost:8000/api/posts/?title=ViewSets&text=&author_id=

נקודת קצה

http://localhost:8000/api/posts/?title=ViewSets

query string

http://localhost:8000/api/posts/?title=ViewSets&text=&author_id=

הכל ביחד

הוספת יכולות סינון

<http://localhost:8000/api/posts/?title=ViewSets>

http://localhost:8000/api/posts/?author_id=1&title=ViewSets

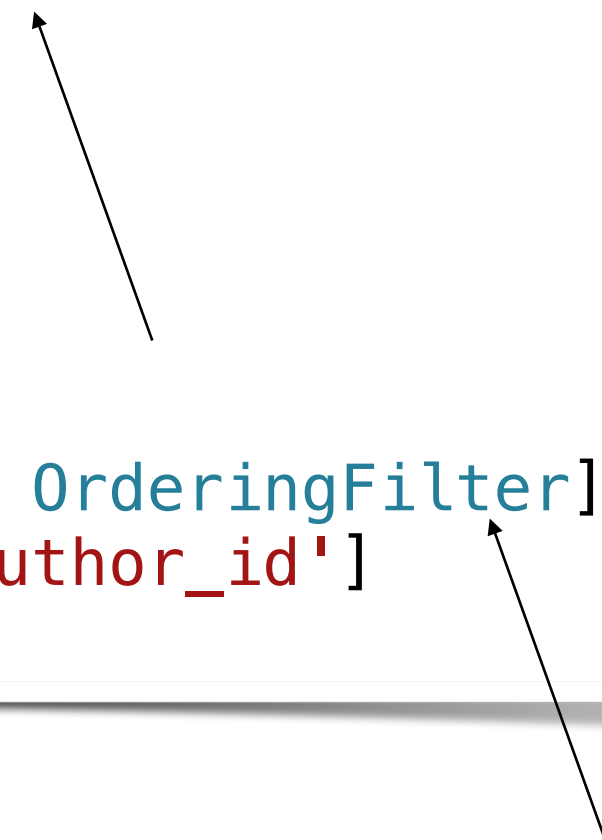
החיפוש הוא מדויק לפי כל האותיות ולפי הCASE

הוספת יכולות מיון

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import OrderingFilter

class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [PostsPermission]

    filter_backends = [DjangoFilterBackend, OrderingFilter]
    filterset_fields = ['title', 'text', 'author_id']
```



Content-Type: application/json
Vary: Accept

```
[
  {
    "id": 6,
    "author_id": 1,
    "title": "Authentication & J",
    "text": "Implementing secure",
    "status": "published",
    "created_at": "2025-09-04T12",
    "updated_at": "2025-09-04T12",
    "tags": []
  },
  {
    "id": 8,
    "author_id": 1,
    "title": "CI/CD Pipelines",
    "text": "Setting up continuo",
    "status": "published",
    "created_at": "2025-09-04T12",
    "updated_at": "2025-09-04T12",
    "tags": []
  },
  {
    "id": 5,
    "author_id": 1,
    "title": "Database Migration",
    "text": "How to handle datab
```

Author:

Ordering

- ID - ascending
- ID - descending
- Title - ascending**
- Title - descending
- Text - ascending
- Text - descending
- Status - ascending
- Status - descending

הוספת יכולות חיפוש

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import OrderingFilter, SearchFilter

class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [PostsPermission]

    filter_backends = [DjangoFilterBackend, OrderingFilter, SearchFilter]
    filterset_fields = ['title', 'text', 'author_id']

    search_fields = ['title', 'text']
```



The diagram consists of two arrows. The first arrow originates from the `SearchFilter` class in the import statement and points to its inclusion in the `filter_backends` list within the `PostViewSet` class. The second arrow originates from the `search_fields` attribute (which is highlighted with a rounded rectangle in the original image) and points to the `SearchFilter` class in the `filter_backends` list, indicating that this attribute is used by the `SearchFilter` to perform searches.

הוספת יכולות חיפוש

/posts/?search=View

za-Net (v. 4.5) - Designing INSTEA... SQL SERVER - @... COLUMNS_UPDA... Decode COLUMN... Using Columns_U... Introduction to SQ... stanf Toolbar Animation... Handling Scrolls w... fitsSystemWindow...

Django REST framework

Api Root / Post List

Post List

GET /api/posts/?search=View

HTTP 200 OK

Allow: GET, POST, HEAD, OPTIONS

Content-Type: application/json

Vary: Accept

```
[
  {
    "id": 1,
    "author_id": 1,
    "title": "ViewSets",
    "text": "After routing has d
    "status": "draft",
    "created_at": "2025-09-04T12
    "updated_at": "2025-09-04T12
    "tags": [
      2
    ]
  }
]
```

Filters

Field filters

Title:

Text:

Author:

Submit

Ordering

title - ascending

title - descending

created_at - ascending

created_at - descending

Search

View

Search

Filters

making sens

Pagination

מחלקים את הדאטה לעמודים
לקוח מקבל עמוד אחד בכל בקשה

settings.py

```
REST_FRAMEWORK = {  
  
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.LimitOffsetPagination',  
    'PAGE_SIZE': 1,  
  
}
```



<http://localhost:8000/api/posts/?limit=3&offset=0>

הלקוח יכול להגדיר את הלימיט בבקשה

↑
כמות התוצאות בעמוד

↑
באיזה אינדקס אנחנו

limit=5

Pagination

settings.py

```
REST_FRAMEWORK = {  
    'DEFAULT_PAGINATION_CLASS':  
    'rest_framework.pagination.PageNumberPagination',  
    'PAGE_SIZE': 2,  
}
```

<http://localhost:8000/api/posts/?page=3>

Pagination

views.py

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import OrderingFilter, SearchFilter
from rest_framework.pagination import PageNumberPagination

class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [PostsPermission]

    filter_backends = [DjangoFilterBackend, OrderingFilter, SearchFilter]
    filterset_fields = ['title', 'text', 'author_id']

    search_fields = ['title', 'text']
    ordering_fields = ['title', 'created_at']
    ordering = ['-created_at']

    pagination_class = PageNumberPagination
```

Cloud Deployment

העלאה לאחסון על שרת באינטרנט.

Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead. For more information on production servers see: <https://docs.djangoproject.com/en/5.2/howto/deployment/>

/Volumes/E4TB/W270225MR/DjangoDRF/finalproject/api/views.py changed, reloading.

Watching for file changes with StatReloader

Performing system checks...

System check identified no issues (0 silenced).

September 09, 2025 – 09:29:05

Django version 5.2.6, using settings 'finalproject.settings'

Starting development server at http://127.0.0.1:8000/

Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead. For more information on production servers see: <https://docs.djangoproject.com/en/5.2/howto/deployment/>

Cloud Deployment

```
pip install gunicorn
```

```
gunicorn finalproject.wsgi
```

Cloud Deployment

כדי להפעיל את גוניקורן צריך לינוקס:

אפשר להתקין ולקבל טרמינל של לינוקס:

[How to install Linux on Windows with WSL](#)

בנתיים אפשר להמשיך להריץ עם:

```
python manage.py runserver
```

והשרת לינוקס יריץ גוניקורן
כי התקנו את זה

Cloud Deployment

הרצה של גוניקורן עם קובץ הגדרות:

 gunicorn.conf.py

```
import multiprocessing
import os

PORT = os.getenv('PORT', '8000')

bind = f"0.0.0.0:{PORT}"
workers = multiprocessing.cpu_count() * 2 + 1
loglevel = 'debug'
timeout = 60
keepalive = 5

# run gunicorn with these settings file
# gunicorn -c gunicorn.conf.py finalproject.wsgi:application
```


Cloud Deployment

הגשה של קבצים סטטיים:

קבצים צריך להגיש באופן שונה מדאטה שמשתנה כל הזמן

<https://whitenoise.readthedocs.io/en/stable/index.html>

pip install whitenoise

QuickStart for Django apps

Edit your `settings.py` file and add WhiteNoise to the `MIDDLEWARE` list, above all other middleware apart from Django's [SecurityMiddleware](#):

```
MIDDLEWARE = [  
    # ...  
    "django.middleware.security.SecurityMiddleware",  
    "whitenoise.middleware.WhiteNoiseMiddleware",  
    # ...  
]
```


Cloud Deployment

whitenoise

```
STORAGES = {  
    # ...  
    "staticfiles": {  
        "BACKEND": "whitenoise.storage.CompressedManifestStaticFilesStorage",  
    },  
}
```

1. Make sure *staticfiles* is configured correctly

If you're familiar with Django you'll know what to do. If you're just getting started with a new Django project then you'll need add the following to the bottom of your `settings.py` file:

```
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Cloud Deployment

whitenoise

```
# Static files (CSS, JavaScript, Images)  
# https://docs.djangoproject.com/en/5.1/howto/static-files/
```

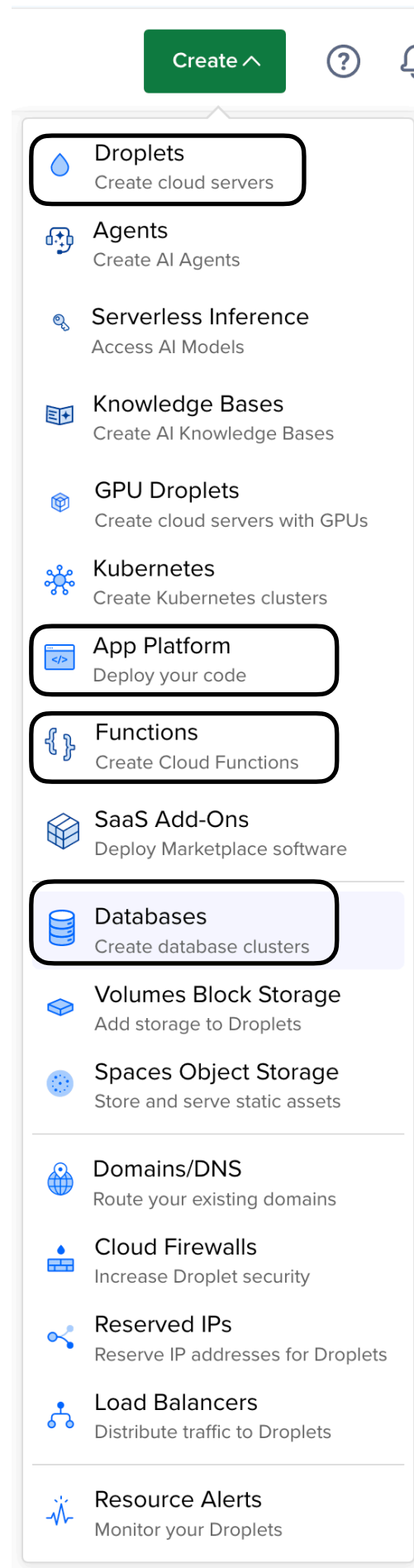
```
STATIC_URL = 'static/'  
STATIC_ROOT = BASE_DIR / "staticfiles"
```

פקודה שאוספת את כל הקבצים הסטטיים מהפרוייקט ומקבצת בתיקיה שתוגש באופן יעיל:

```
$ python manage.py collectstatic
```

python manage.py collectstatic

VPS



Digital Ocean

droplet = מחשב שלי בענן
VPS

App Platform

שירות PaSS

שדיגיטל מזהים את הפרוייקט שלכם מגיט ומריצים אותו בענן

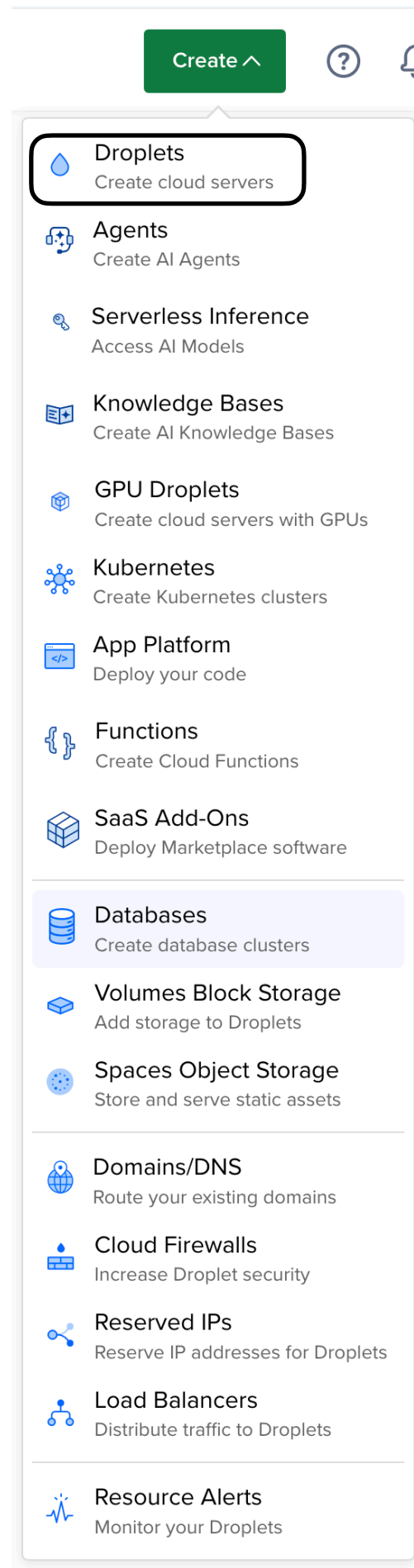
Cloud Functions

כותבים קוד ביחידות של פונקציות
לכל פונקציה יש כתובת בענן

ארכיטקטורה של מיקרו-סרביסים

Database

דטהבייס



Digital Ocean

ניצור Droplet:

Digital Ocean

בהגדרות של הDroplet נבחר בDjango עם אובונטו:

Choose an image

OS Marketplace (253) Custom images

🔍 Django

[Explore all Marketplace Solutions](#)



OpenLiteSpeed Django

Frameworks

A lightweight app which automatically installs OpenLiteSpeed and Django, for an image that can be up to 2.5 times fas...

django

Django

Frameworks

A high-level Python Web framework

django

Django 3.2 LTS

Frameworks

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design.

⬆️ ⬆️ navigate ↶ go ⌨️ esc exit

Our scalable database cluster service includes daily backups with PITR, automated failover, and end-to-end SSL.

Recommended for you



WordPress on Ubuntu 22.04

[Details](#)

plesk Plesk 18.0.68 on Ubuntu 24.04

[Details](#)



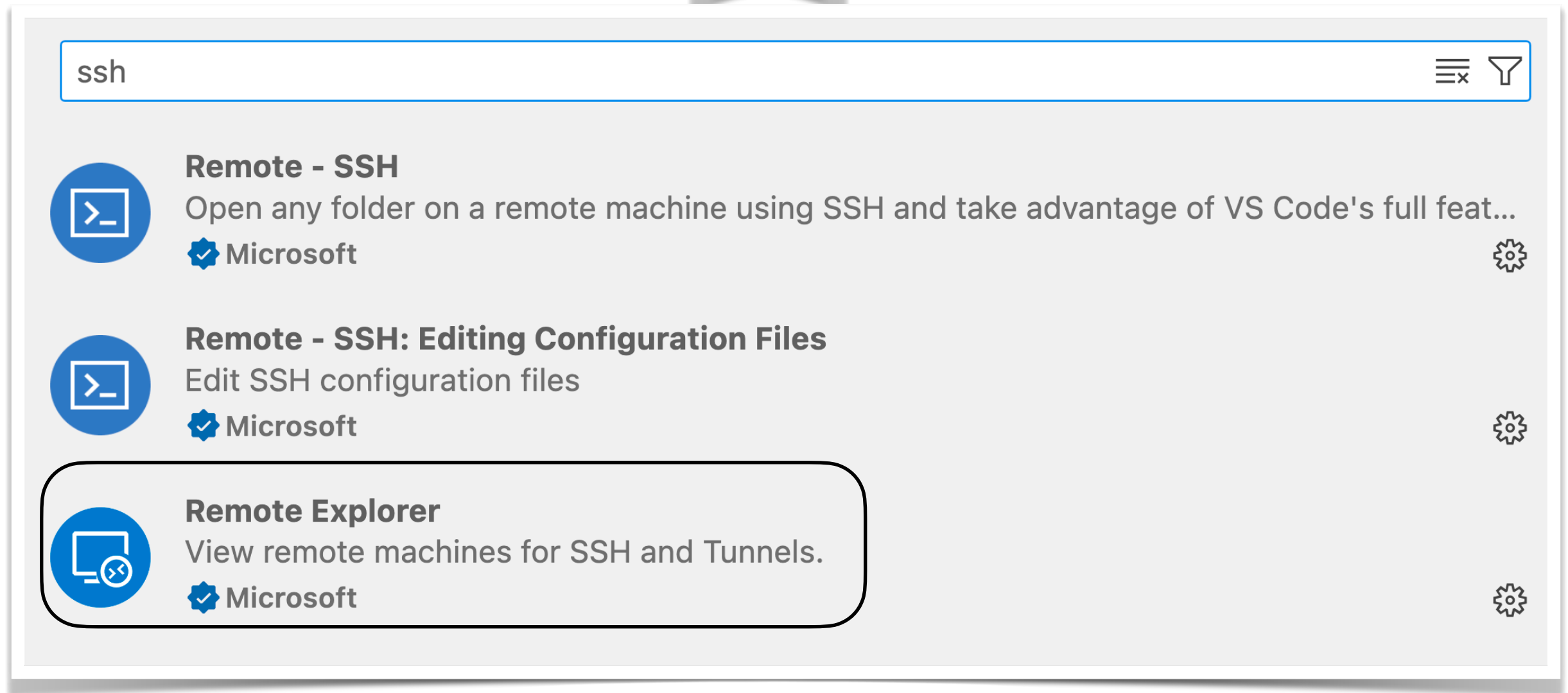
Docker on Ubuntu 22.04

[Details](#)

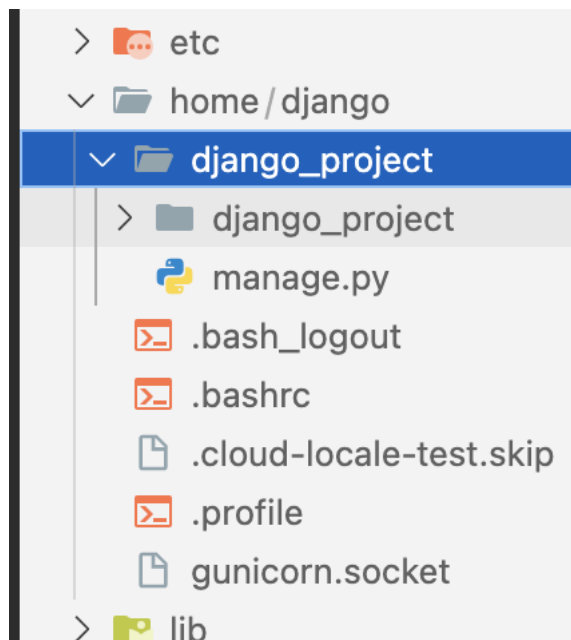
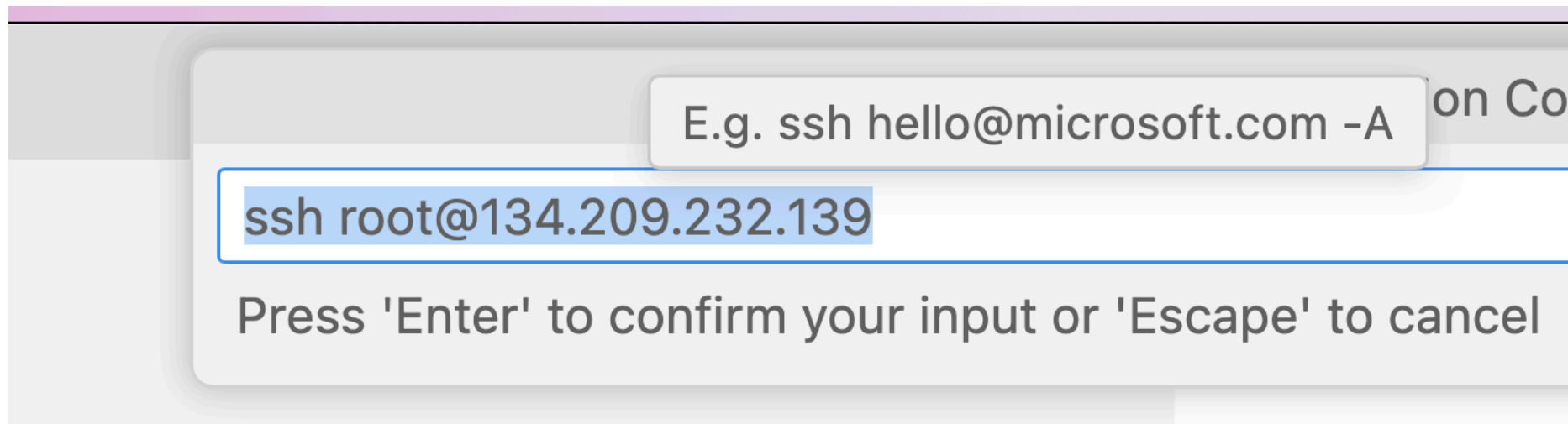
תוספים לעבודה עם מחשב מרוחק:

התקנת תוספים:

דרישות:
כתובת ip
וסיסמא



תוספים לעבודה עם מחשב מרוחק:

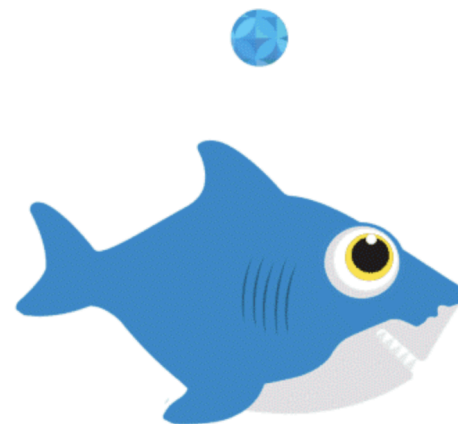


זה המסך שעולה כשנכנסים לכתובת IP של Droplet:

<http://134.209.232.139>

על הDroplet
מותקן גם דטהבייס של פוסטגרס
הסבר על איך מקבלים את הסיסמא
בעמוד הבא

Sammy welcomes you to your Droplet!



Things to do with this script

This message is coming to you via a simple Django application that's live on your Droplet! This droplet is all set up with Python, Django, and Postgres. It's also using Gunicorn to run the application on system boot and using nginx to proxy traffic to the application over port 80.

Get your code on here

- SSH into your Droplet.

```
ssh root@134.209.232.139
```

הוראות התחברות:

- Provide the password that was emailed to you. You can also use an SSH key, if you selected that option during Droplet creation, by [following these instructions](#).
- Note the login message, it has important details for connecting to your Postgres database, among other things!
- `git clone` your code onto the droplet. You can try to reuse this project, located in `/home/django/django_project`, or start fresh in a new location and edit Gunicorn's configuration to point to it at `/etc/systemd/system/gunicorn.service`. You can also change how nginx is routing traffic by editing `/etc/nginx/sites-enabled/django`
 - Note: If you're not using a source control, you can [directly upload the files to your droplet using SFTP](#).
- `cd` into the directory where your Python code lives, and install any dependencies. For example, if you have a `requirements.txt` file, run `pip install -r requirements.txt`.
- That's it! Whenever you make code changes, reload Gunicorn like so:

```
PID=$(systemctl show --value -p MainPID gunicorn.service) && kill -HUP $PID
```

Play in the admin area

הוראות הפעלה מחדש
של השרת לאחר שינויי קוד:

המשך מעמוד קודם:



Things to do with this script

This message is coming to you via a simple Django application that's live on your Droplet! This droplet is all set up with Python, Django, and Postgres. It's also using Gunicorn to run the application on system boot and using nginx to proxy traffic to the application over port 80.

Get your code on here

- SSH into your Droplet.

```
ssh root@134.209.232.139
```

- Provide the password that was emailed to you. You can also use an SSH key, if you selected that option during Droplet creation, by [following these instructions](#).
- Note the login message, it has important details for connecting to your Postgres database, among other things!
- `git clone` your code onto the droplet. You can try to reuse this project, located in `/home/django/django_project`, or start fresh in a new location and edit Gunicorn's configuration to point to it at `/etc/systemd/system/gunicorn.service`. You can also change how nginx is routing traffic by editing `/etc/nginx/sites-enabled/django`
 - Note: If you're not using a source control, you can [directly upload the files to your droplet using SFTP](#).
- `cd` into the directory where your Python code lives, and install any dependencies. For example, if you have a `requirements.txt` file, run `pip install -r requirements.txt`.
- That's it! Whenever you make code changes, reload Gunicorn like so:

```
PID=$(systemctl show --value -p MainPID gunicorn.service) && kill -HUP $PID
```

Play in the admin area

The standard Django admin area is accessible at [/admin](#). The login and password are stored in the `DJANGO_USER*` values you see when you call `cat /root/.digitalocean_passwords` while logged in over SSH.

Get production-ready

- [Set up a non-root user for day-to-day use](#)
- [Set up a DigitalOcean cloud firewall](#) (they're free).
- [Register a custom domain](#)
- Have data needs? You can mount a [volume](#) (up to 16TB) to this server, or use a [Space](#), which is an S3-compatible bucket for storing objects.

פקודה להרצה בטרמינל -
הפקודה תציג את הסיסמאות
לדטהבייס

פקודה להפעלה מחדש של
גוניקורן לאחר שינוי קוד